

Typing math into MathCompile — input reference

How to type the math a typical undergraduate needs into `<math>-field</math>` cells. Companion to `SYMBOLS.md` (the full command inventory) and `MISSING-INPUT-FEATURES.md` (what *can't* be typed, or misbehaves).

Every keystroke sequence below was verified by driving real cells in the dev app (`npm run dev` , port 5573) with synthetic keyboard input — ~800 probes — and recording the serialized LaTeX the cell produces. Worked examples are taken from three Harvard undergraduate theses: Buller (knot theory), Fogelson (bifurcation theory/ODEs), Salkinder (knot theory / low-dimensional topology). Verified against `main @ 4f65954` ; updated for the autocomplete menu/picker, `\name` environment shortcuts, `\array{spx}` arg input, and `\def (@ 6e4bf04)`.

Notation used here: $\frac{\quad}{\quad}$ +Enter means "type the word after a backslash, press Enter to accept it".

Right / Tab / Up / Down are arrow and tab keys. Quoted strings are typed literally.

The six rules that govern everything

1. **** ** opens the command input.** Type `\` then a word, accept with **Enter** (or Space, or any non-letter). Accepting inserts the symbol or command. Space after a command word also accepts it (`\alpha` → `\alpha + space`). As you type a **suggestion dropdown** lists matching commands — arrows + Enter accept the highlight, Esc dismisses it until the next keystroke. A plain word-run of 2+ letters suggests too (`sq` + Enter → `\sqrt`). **Ctrl+Space** opens the symbol picker: type to filter, Enter inserts the highlighted card, Esc closes.
2. **Blocks stay open — this is the #1 gotcha.** The caret stays *inside* a subscript, superscript, fraction half, root, operator bound, accent, matrix cell, or `\text` argument until you press **Right** or **Tab**. Everything you type meanwhile lands inside the block — including `+`, `-`, `,`, `*`, spaces, parens, and even `_ / ^ / /` (which nest):
 - `x_1+2` → `x_{1+2}` ✗ (you wanted `x_{1}+2`)
 - `x^2-3` → `x^{2-3}` ✗
 - `v_1,v_2,v_k` → `v_{1,v_{2,v_{k}}}` ✗ (the `,` is inside, next `_` nests)
 - `x^2 y` → `x^{2\ y}` ✗ (space inside a sup is a `\ gap`)
 - `x_1(t)` → `x_{1\left(t\right)}` ✗
 - `e^x^2` → `e^{x^{2}}` (nested sup — not `e^{x^2}` ambiguity, it literally nests) Always **Right** (or **Tab**) to close a block before typing what follows it. `T_p,q` → `T_{p,q}` is the lucky exception where a comma inside a sub is what you wanted anyway.
3. **Auto-pairs.** (`[ { |` insert a `\left...\right` pair; the caret sits inside. Typing the same closer hops over it; typing a *different* closer still closes it — `(0,1]` → `\left(0,1\right]` for free. `</>` do **not** pair — use `\angle`, which auto-pairs to `\left\angle...\right\angle`. **Do not type `\rangle` (or `\rVert`)** to close it — the close is already there; typing it swaps the close glyph to `\angle` and produces latex that fails to re-parse (blank cell on reload). Exit with Right/Ctrl+End instead.
4. **Enter vs Shift+Enter.** At top level **Enter** inserts a `\displaylines` line break *in the same cell*; **Shift+Enter** makes a **new cell**. Inside an open `\command`, Enter accepts the command. Inside a matrix/environment, Enter adds a **row** (keeping the column) and Shift+Space adds a **column**. Inside a script/fraction, Enter snaps the caret out to the atom first, then breaks the line.
5. **Smart mode** (the default toggle in the header) adds two things on top of plain mode: `autoCommands` — the words `int iint antid sum sqrt prod pi infy theta derivative def work` with no backslash — and `autoSubscriptNumerals` (`x2` → `x_{2}`). Upright operator names (`sin`, `lim`, `det`, ...) and unicode symbol typing are on in **both** modes.
6. **Unicode input works.** Typing $\alpha \beta \pi \theta \infty \leq \geq \neq \approx \rightarrow \mapsto ^\circ \sqrt{\int \prod \sum \coprod \forall \exists \in \notin \subset \subseteq \cup \cap \wedge \vee \neg \rightarrow \Rightarrow \langle \rangle \| \mathbb{R} \mathbb{Z} \mathbb{N} \mathbb{Q} \mathbb{C} \pm \times \div -}$ directly inserts the symbol — no backslash needed ($\mathbb{R}\mathbb{Z}\mathbb{N}\mathbb{Q}\mathbb{C}$ become `\mathbb{...}`), `\int` → `\iint`). Caveats: `\int/\sum` insert the *bounded* templates (caret lands in the lower bound), `\sqrt{\sum/\prod}` open their structure while inside a bound, `\langle\rangle\|` stay literal unicode chars (a usable workaround for the `\rangle` bug in rule 3), `\approx` stays a literal `\approx`, and the unicode prime `'` stays unicode — use ASCII `'`.

Scripts, primes, factorials

Want	Type	Get
x_1	<code>x_1</code>	<code>x_{1}</code>
x_{12}	<code>x_12</code>	<code>x_{12}</code> (subscript grabs all digits)
$x_{\max}$	<code>x_max</code>	<code>x_{\max}</code> (auto-operator inside sub)
x^2	<code>x^2</code>	<code>x^{2}</code>
A^{-1}	<code>A^-1</code>	<code>A^{-1}</code>
$t^{1/2}$	<code>t^1/2</code>	<code>t^{\frac{1}{2}}</code> (/ inside a sup makes a nested frac)
x_1^2	<code>x_1 Right ^2</code>	<code>x_1^{2}</code>
prescript i x	<code>_i Right x</code>	<code>_i x</code> — without Right, <code>_ix</code> → <code>_ix</code> nests
$e^{i\pi}$	<code>e^(i \pi +Enter)</code>	<code>e^{\left(i\pi\right)}</code>
f', f'', f'''	<code>f' / f'' / f'''</code>	<code>f', f'', f'''</code>
g'_x	<code>g'_x</code>	<code>g'_{x}</code>
$\prime$	<code>\dprime +Enter</code>	$\prime$ (unicode glyph)
$n!, n!!$	<code>n! / n!!</code>	<code>n!, n!!</code>
x_2 (smart mode)	<code>x2</code>	<code>x_{2}</code> — autoSubscriptNumerals. Trap: <code>d3</code> in <code>(mod3)</code> becomes <code>d_{3}</code> ; smart off gives <code>x2</code> literal

A lone subscript like `x_i` is arrow-skippable at baseline (Right hops over it cleanly). Backspace on a bound deletes the whole bound before descending into it. `{...}` inside a script groups invisibly: `x_{i+1}` → `x_{i+1}`.

Fractions and binomials

`/` is the fast path: it grabs the atoms to the *left* since the last binary operator and builds a `\frac`.

Want	Type	Get
$\frac{1}{2}$	<code>1/2</code>	<code>\frac{1}{2}</code> — caret lands in denominator
$\frac{x}{2}$	<code>x/2</code>	<code>\frac{x}{2}</code>
$x + \frac{y}{2}$	<code>x+y/2</code>	<code>x+\frac{y}{2}</code> — <code>/</code> stops at the <code>+</code>
$\frac{a+b}{2}$	<code>(a+b)/2</code>	<code>\frac{\left(a+b\right)}{2}</code>
empty frac template	<code>\frac +Enter</code>	caret in numerator; Tab → denominator; Right/Tab → baseline
$\frac{a+b}{c+d}$	<code>\frac +Enter a+b Right c+d</code>	<code>\frac{a+b}{c+d}</code>
$\frac{1}{2/3}$	<code>1/2/3</code>	<code>\frac{1}{\frac{2}{3}}</code> (nested, right-associative)
$\frac{a}{b}$ infix	<code>a\over +Enter b</code>	<code>\frac{a}{b}</code>
$\frac{a}{b}$ infix	<code>a\atop +Enter b</code>	<code>a\atop b</code> (serialize stays <code>\atop</code>)
$\binom{n}{k}$	<code>\binom +Enter n Tab k</code>	<code>\binom{n}{k}</code>
$\binom{n}{k}$ infix	<code>n\choose +Enter k</code>	<code>\binom{n}{k}</code>
$\binom{n}{k}$ display/text	<code>\dbinom +Enter n Tab k</code>	serializes <code>\binom{n}{k}</code> — <code>\tbinom</code> same

Aliases that serialize to `\frac`: `\dfrac`, `\cfrac`, `\fraction`. Inline slash instead of a fraction: `a\slash +Enter b` → `a/b`. `\tfrac`, `\genfrac` are missing (see MISSING-INPUT-FEATURES.md).

Roots

Want	Type	Get
$\sqrt{x}$	<code>sqrt</code> (smart) or <code>\sqrt</code> +Enter	<code>\sqrt{x}</code> — caret in the radicand, Right/Tab exits
$\sqrt[3]{x}$	<code>\nthroot</code> +Enter 3 Tab x	<code>\sqrt[3]{x}</code> — index first, Tab → radicand
$\sqrt[3]{x}$	<code>\cbrt</code> +Enter x	<code>\sqrt[3]{x}</code>
$i = \sqrt{-1}$	<code>i=sqrt -1</code>	<code>i=\sqrt{-1}</code>

Trap: `\sqrt` then `[3]` gives `\sqrt{\left[3\right]x}` — brackets inside a radicand are literal; the index only exists via `\nthroot` / `\cbrt`. Smart mode off: `sqrt` is the literal letters `sqrtx`; use `\sqrt` +Enter.

Greek letters

`\` +word+Enter for every Greek letter, lower and capital: `\alpha` `\beta` `\gamma` `\Gamma` `\delta` `\Delta` `\epsilon` `\zeta` `\eta` `\theta` `\Theta` `\iota` `\kappa` `\lambda` `\Lambda` `\mu` `\nu` `\xi` `\Xi` `\pi` `\Pi` `\rho` `\sigma` `\Sigma` `\tau` `\upsilon` `\Upsilon` `\phi` `\Phi` `\chi` `\psi` `\Psi` `\omega` `\Omega`.

Variants: `\varepsilon` (also `\epsiv`), `\varphi` (`\phiv`), `\vartheta` (`\thetasym`, `\thetav`), `\varrho` (`\rhov`), `\varsigma` (`\sigmaf`, `\sigmav`), `\varkappa` (`\kappav`), `\varpi` (`\piv`), `\digamma` (`\gammad`).

Smart mode: bare `pi` → `\pi` and `theta` → `\theta` with no backslash. Bare `alpha` stays the literal letters `alpha` — only `pi` / `theta` are smart commands. Typing the unicode chars `αβπθ` directly also works.

Named functions and upright operators

Auto-operator names (upright, both modes, no backslash needed) — type the word then the argument:

```
sin cos tan arcsin arccos arctan sinh cosh tanh sec csc cot coth
ln log lg exp det dim ker gcd hom arg deg Pr injlim projlim
min max sup inf lim limsup liminf
gcf hcf lcm proj span ctg cosec → serialize as \operatorname{...}
arctanh and other generated arc/h/arh/arch variants of sin cos tan sec
cosec csc cotan cot ctg → \operatorname{...}
```

`sin(x)` → `\sin\left(x\right)`, `gcd(a,b)` → `\gcd\left(a,b\right)`. `\sin` +Enter works too. Bounds weld like subscripts and stay open: `log_2x` → `\log_{2}x` (the `x` is *inside* the sub) — for `\log_{2}x` type `log_2` Right `x`.

Custom operator names: `\operatorname` itself is dropped (typed `\operatorname{tr}` +Enter produces italic `tr`) — but `\mathop{...}` **works:** `\mathop` +Enter `tr` Right (A) → `\mathop{tr}\left(A\right)` which renders upright. Use it for `tr`, `Aut`, `Hom`, `GL`, etc.

Mid-word trap: these fire anywhere inside a word, so `crossings` → `cro\sin gs`, `single` → `\sin gle`, `cosine` → `\cos ine`, `info` → `\inf o`, `minimum` → `\min imum`, `elim` → `e\lim`, `asin` → `a\sin`. Workaround: `\text{crossings}` +Enter for words, or use a symbol (`C`).

Function definitions

`f(x)` reads as `f * x` — parens are a factor, not a signature. Mark a definition with `\def` +Enter, which inserts `\text{def}` (text block + trailing space, caret ready for the signature):

`\def` +Enter `g(x)=2x` → `\text{def} g\left(x\right)=2x`, and codegen emits `def g(x): return 2*x`. `def` is also a smart-mode word — bare `def` +Enter works with no backslash.

Derivatives

Want	Type	Get
$\frac{dy}{dx}$	dy/dx	<code>\frac{dy}{dx}</code>
$\frac{d}{dx}$	d/dx	<code>\frac{d}{dx}</code>
$\frac{df}{dx}$ via command	<code>\derivative</code> +Enter x Up f	expands at insertion to <code>\frac{d}{dx}</code> atoms, caret in denominator → <code>\frac{df}{dx}</code>
$\frac{d}{dx} f$	<code>\derivative</code> +Enter x Right f	<code>\frac{d}{dx}f</code>
$\frac{\partial F}{\partial x}$	<code>\frac</code> +Enter <code>\partial</code> +Enter F Right <code>\partial</code> +Enter x	<code>\frac{\partial F}{\partial x}</code>
$\dot{x}, \ddot{x}$	<code>\dot</code> +Enter x Right / <code>\ddot</code> +Enter x Right	<code>\dot{x}</code> , <code>\ddot{x}</code>
$x' = F(x, \mu)$ (Fogelson)	<code>x'=F(x,\mu)</code>	<code>x'=F\left(x,\mu\right)</code>

`dIsDerivative` off makes `\derivative` expand to `D( )` instead.

Integrals

Want	Type	Get
$\int_a^b x^2 dx$	<code>\int</code> +Enter a Tab b Tab <code>x^2</code> Tab dx	<code>\int_{a}^{b}x^{2}dx</code> — caret lands in the lower bound; Tab walks lower → upper → baseline
same, smart	<code>int</code> a Tab b Tab <code>x^2</code> Tab dx	identical
skip the upper bound	<code>\int</code> +Enter a Right Right x	<code>\int_{a}^{ }x</code> — Right×2 hops past the empty upper
$\iint_D f$ boundless	<code>iint</code> or <code>\iint</code> +Enter <code>_D</code> Right f(x)	<code>\iint_{D}f\left(x\right)</code> — <code>_ / ^</code> after it weld a plain supsub
$\int$ indefinite	<code>antid</code>	<code>\antid x^{2}dx</code> — insertion alias for <code>\int</code> at compile time
$\iiint \int \dots \int \int$	<code>\iiint</code> +Enter etc.	boundless signs; weld bounds with <code>_ / ^</code>
$\oint_C$	<code>\oint</code> +Enter <code>_C</code> Right	<code>\oint_{C}</code>
unicode	<code>\int / \iint</code> char	<code>\int_{ }^{ }</code> bounded / <code>\iint</code> boundless

Trap: after `int` / `sum` / `prod` smart words the caret is already inside the lower bound — do **not** type `_`: `int _a ^b x dx` → `\int_{_a}^{b}xdx` } garbage.

Sums, products, big operators

Want	Type	Get
$\sum_{i=1}^n i^2$	<code>\sum</code> +Enter i=1 Right n Right <code>i^2</code>	<code>\sum_{i=1}^{n}i^{2}</code>
$\sum_{i=1}^{\infty} x^i$	<code>\sum</code> +Enter i=1 Right <code>\infty</code> +Enter Right <code>x^i</code>	<code>\sum_{i=1}^{\infty}x^{i}</code> — use <code>\infty</code> +Enter inside bounds ; bare <code>\infty</code> in a bound → <code>\inf ty</code>
$\sum_c$	<code>\sum</code> +Enter c Right Right	<code>\sum_{c}^{ }</code> then keep typing
$\prod$	<code>prod</code> or <code>\prod</code> +Enter	<code>\prod_{ }^{ }</code> bounded template

Want	Type	Get
$\bigcup_{i=1}^n K_i$	<code>\bigcup +Enter _i=1 Right ^n Right K_i</code>	<code>\bigcup_{i=1}^{n}K_{i}</code> — bigops are boundless; <code>_ / ^</code> weld bounds
$\bigcap, \bigoplus, \bigotimes, \bigvee, \bigwedge, \bigcup, \bigcap, \bigoplus, \bigotimes, \bigvee, \bigwedge$	<code>\bigcap +Enter etc.</code>	boundless signs; weld bounds with <code>_ / ^</code>
$\sum_{c \in X}$	<code>\underset +Enter c \in +Enter X Tab \sum +Enter</code>	<code>\underset{c\in X}{\sum_{ }{ }}</code> — arg1 = below-text, arg2 = base
$\overset{\text{def}}{=}$	<code>\overset +Enter def Tab =</code>	<code>\overset{def}{=}</code> — <code>\stackrel</code> +Enter is an alias
multi-line bound	<code>\sum +Enter \substack +Enter i=1 Enter j=2</code>	<code>\sum_{\substack{i=1\ j=2}}{ }</code> — Enter inside <code>\substack</code> adds a bound row
$\sum, \int$	<code>\limits / \nolimits +Enter</code> after the sign	serialize as a trailing leaf (<code>\sum_{i=1}^{ }\limits</code>)

Aliases: `\sum`, `\product`, `\integral`, `\coproduct`.

Limits and bound-taking operators

`\lim`, `\limsup`, `\liminf`, `\sup`, `\inf`, `\max`, `\min` are **boundless** auto-operators — there is no bound template; type `_` to weld the condition:

Want	Type	Get
$\lim_{x \rightarrow 0} f(x)$	<code>\lim _x->0 Right f(x)</code>	<code>\lim_{x\to 0}f\left(x\right)</code> — <code>-></code> becomes <code>\to</code> for free
$\limsup_n a_n$	<code>\limsup +Enter _n Right a_n</code>	<code>\limsup_{n}a_{n}</code>
$\sup_x, \inf_x, \max_i, \min_s$	<code>\sup_x Right ...</code>	<code>\sup_{x}</code> etc.
$a_n \downarrow L$ (converges)	<code>a_n Right \converges +Enter L</code>	<code>a_{n}\downarrow L</code> — Desmos semantics: <code>\converges</code> → <code>\downarrow</code> , <code>\diverges</code> → <code>\uparrow</code> (without the Right, <code>a_n\converges +Enter L</code> → <code>a_{n}\downarrow L</code> , arrow inside the sub)

`\lim` +Enter alone gives boundless `\lim`; the `_` after it welds.

Relations and inequalities

Typing shortcuts: `<=` → `\le`, `>=` → `\ge`, `->` → `\to` (all auto). `=>`, `!=`, `<>` stay **literal** — use `\Rightarrow`, `\ne`.

Commands (all `\word +Enter`): `\ne`, `\neq`, `\approx`, `\asymp`, `\sim`, `\simeq`, `\cong`, `\equiv`, `\propto`, `\doteq`, `\ll`, `\gg`, `\prec`, `\preceq`, `\succ`, `\succeq`, `\models`, `\vdash`, `\dashv`, `\perp`, `\parallel`, `\mid`, `\in` (`\isin`), `\ni` (`\contains`), `\subset` (`\sub`), `\subteq` (`\sube`, `\subeq`, `\subete`), `\supset` (`\sup`), `\supseteq` (`\supe`, `\supseq`, `\supsete`), `\sqsubset`, `\sqsubteq`, `\sqsupset`, `\sqsupseteq`.

AMS relations now work: `\approxeq`, `\lesssim`, `\gtrsim`, `\leqslant`, `\thickapprox`, `\risingdotseq`, `\fallingdotseq`, `\subsetneq`, `\supsetneq`, `\Subset`, `\Supset`, `\subteqq`, `\eqcirc`, `\circeq`, `\bumpeq`, `\Bumpeq`, `\backsimeq`, `\between`, `\Join`, `\shortmid`, `\shortparallel`, `\precsim`, `\succsim`, `\vdash`, `\Vdash`, `\triangleq`, `\coloneq`, `\coloneqq`, `\colonequals`, `\eqqcolon`.

Definitional equals `:=`: type `:=` literally, or `\assign` / `\coloneqq` / `\colonequals` (all serialize `\coloneqq`). For `def` `=` use `\overset +Enter def Tab =` or `\stackrel +Enter`.

Negations — `\not` is a **no-op prefix**. Typing `\not +Enter` before `=`, `\in`, `\sim`, `\mid`, `\subset` inserts *nothing* (`x\not=` → `x=y`). Use the dedicated `\n...` commands, which serialize as `\not X`: `\nless`, `\ngtr`, `\nleq`, `\ngeq`.

```
\nsim \ncong \nparallel \nsub \nsupe \nsubset \nsubseteq \nsubseteqeq \nsup \nsupe \nsupset \nsupseteq
\nsupseteq \nmid \nprec \nsucc \nvDash \notin (\notin \nnotin \notincontains \doesnotcontain for \ni)
\nexists.
```

Still missing: `\pitchfork`, `\varpropto`, `\eqdef`, `\questeq`, `\hateq`, `\veeeq` — see MISSING-INPUT-FEATURES.md.

Arrows

Want	Type	Get
$x \rightarrow y$	<code>x->y</code> or <code>x\to +Enter y</code>	<code>x\to y</code>
$x \mapsto y$	<code>x\mapsto +Enter y</code>	<code>x\mapsto y</code> — <code>**x</code>
$f: X \twoheadrightarrow Y$ surjection	<code>\twoheadrightarrow +Enter</code>	<code>f:X\twoheadrightarrow Y</code> — Salkinder's <code>p:G(K)\rightarrow D_{2n}</code>
$x \rightharpoonup y$ inclusion	<code>\rightarrowtail / \leftarrowtail /</code> <code>\twoheadleftarrow</code>	
labelled arrow	<code>\xrightarrow +Enter f</code>	<code>\xrightarrow{f}</code> — <code>\xleftarrow</code> same; single label arg only
$\Rightarrow \Leftrightarrow \Leftarrow$	<code>\Rightarrow</code> or <code>\implies</code> / <code>\Leftrightarrow</code> or <code>\iff</code> / <code>\Leftarrow</code> or <code>\impliedby</code>	<code>\implies / \iff / \impliedby</code> serialize literally (render the same)
long/hook/harpoon	<code>\longrightarrow \Longrightarrow \longleftarrow</code> <code>\Longleftarrow \longleftarrowrightarrow</code> <code>\Longleftarrowrightarrow \hookrightarrow \hookleftarrow</code> <code>\leftharpoondown \leftharpoonup \rightharpoondown</code> <code>\rightharpoonup</code>	all serialize
AMS arrows	<code>\rightsquigarrow \leadsto \looparrowright</code> <code>\looparrowleft \curvearrowright \curvearrowleft</code> <code>\circlearrowright \circlearrowleft \Rsh \Lsh</code> <code>\dashrightarrow \dashleftarrow \Lleftarrow</code> <code>\Rightarrow \upharpoonright \upharpoonleft</code> <code>\downharpoonright \downharpoonleft</code> <code>\rightleftharpoons \leftrightharpoons</code>	all serialize
directions	<code>\uparrow \downarrow \updownarrow \Uparrow</code> <code>\Downarrow \Updownarrow \nearrow \nrightarrow \searrow</code> <code>\swarrow</code>	
misc	<code>\gets \rightarrow, \leftarrow (\larr) \rightarrow</code> <code>(\rarr) \leftrightarrow (\lrrarr \harr), capital \lArr</code> <code>\rArr \lRarr \hArr</code>	aliases
restriction $\upharpoonright$	<code>\upharpoonright +Enter</code>	<code>f\upharpoonright_{\{S\}}</code> — or <code>f</code> <code>\vert +Enter _S \rightarrow f</code>

Missing: `\longmapsto`, `\mapsfrom`, `\multimap`, `\nrightarrow`, `\nRightarrow`.

Sets, intervals, delimiters

Want	Type	Get
$\{1, 2, 3\}$	<code>{1,2,3}</code>	<code>\left\{1,2,3\right\}</code> (auto-pair)
$\{x \mid x > 0\}$	<code>{x\mid +Enter x>0}</code>	<code>\left\{x\mid x>0\right\}</code> — cleanest
$\{x : x > 0\}$	<code>{x:x>0}</code>	<code>\left\{x:x>0\right\}</code> — colon also fine

Want	Type	Get
$\{x x > 0\}$ pipe	<code>`{x</code>	<code>x>0}`</code>
$(0,1) [0,1) (-\infty,t]$	typed literally	<code>\left(0,1\right) \left[0,1\right)</code> <code>\left(-\infty,t\right]</code> — - then <code>\infty</code> +Enter
$\in \notin$	<code>\in / \notin / \ni</code>	
$\subset \subseteq \supset \cup \cap \backslash \setminus$	<code>\subset \subseteq \subsetneq</code> <code>\supset \supsetneq \cap</code> <code>\setminus \backslash</code> <code>\smallsetminus</code>	<code>\union / intersect / intersection</code> → <code>\cup / \cap</code>
$\emptyset$	<code>\emptyset</code> or <code>\varnothing</code>	<code>\varnothing</code>
$ x $	<code>`</code>	<code>x</code>
non-paired $ $	<code>\vert</code> +Enter	bare <code>`</code>
$\ x\ $	<code>\lVert</code> +Enter <code>x</code> then Right/Ctrl+End	<code>\left\lVert x\right\rVert</code> — do not type <code>\rVert</code> (it swaps the close to <code>\lVert</code> , see rule 3). <code>`</code>
$\langle x,y x^p = y^q \rangle$	<code>\langle</code> +Enter <code>x,y\mid</code> +Enter <code>x^p</code> Right <code>=y^q</code> , exit with Ctrl+End	<code>\left\langle x,y\mid</code> <code>x^{\text{p}}=y^{\text{q}}\right\rangle</code> — do not type <code>\rangle</code> ; also inside <code>\langle \dots \rangle</code> raw <code>`</code>
$\lfloor \rfloor$	<code>\lfloor</code> +Enter <code>x</code> <code>\rfloor</code> +Enter ...	plain delimiters (no <code>\left</code>)
eval bar $x^2 _1^2$	<code>x^2</code> Right <code>\vert</code> +Enter <code>_1</code> Right <code>^2</code>	<code>x^{\text{2}}\vert_{\text{1}}^{\text{2}}</code> — clean, no trailing pair close
<code>\left(</code> + any closer	<code>\left</code> +Enter then <code>([{</code> or <code>`</code>	<code>`</code>
<code>\big / \Big / \bigg / \Bigg</code>	<code>\big</code> +Enter then delimiter	flaky — produced inconsistent results in probes; <code>\left...\right</code> is reliable

Binary operators

`\pm \mp \times \cdot \div \circ \ast \oplus \otimes \ominus \odot \sqcup \uplus \amalg \wedge \vee \sqcap \wr \diamond \bullet \dagger \ddagger \bowtie \triangleleft \triangleright \frown \smile \bigcirc \backslash \setminus \smallsetminus \bigstar \circledR \circledS` — all `\word` +Enter.
(`\dagger / \ddagger` serialize as `\dag / \ddag`.)

Typing `*` gives `\cdot`. Aliases: `\land / \and` → `\wedge`, `\lor / \or` → `\vee`, `\star / \loast / \lowast` → `\ast`,
`\cross` → `\times`, `\sdot` → `\cdot`, `\circle` → `\circ`, `\bullet / \bull` → `\bullet`, `\circledot` → `\odot`,
`\union` → `\cup`, `\intersect` → `\cap`, `\plusminus` → `\pm`, `\minusplus` → `\mp`.

Wrong-symbol bugs (registered but serialize to the wrong thing): `\oslash`, `\0`, `\o`, `\0slash` → `\varnothing`;
`\divides` → `\div` (should be `|`); `\converges` → `\downarrow`, `\diverges` → `\uparrow`; `\P` → `\mathbb{P}` (should
be `\P`).

Logic and quantifiers

`\forall \exists (\exist \xist \xists) \nexists (\nexist) \neg \therefore (\therefor) \because (\cuz)`
`\top \bot \models \vdash \dashv \Vdash \Vdash` — all standard. `p\land` +Enter `q\lor` +Enter `r` →
`p\wedge q\vee r` (the `\land / \lor` aliases serialize to `\wedge / \vee`). `\not` +Enter inserts nothing — use
`\nexists / \ne` etc. (see Relations).

Accents and decorations

Want	Type	Get
$\hat{x}$ $\hat{A}B$	<code>\hat</code> +Enter <code>x</code>	<code>\hat{x}</code>
$\vec{v}$ $\vec{AB}$	<code>\vec</code> +Enter <code>v</code>	<code>\vec{v}</code>
$\overline{x, z_1 z_2}$	<code>\overline</code> +Enter <code>z_1 z_2</code> Right	<code>\overline{z_{1z_2}}</code> — accent args stay open: <code>\bar</code> +Enter <code>x+y</code> swallows the <code>+</code> (<code>\overline{x+...}</code>), Right exits
$\tilde{H}$ $\widetilde{abc}$	<code>\tilde</code> +Enter / <code>\widetilde</code> +Enter <code>abc</code> Right	<code>\tilde{H}</code> , <code>\widetilde{abc}</code>
$\underline{x}$, $\dot{x}$, $\ddot{x}$	<code>\underline</code> +Enter / <code>\dot</code> +Enter / <code>\ddot</code> +Enter <code>x</code> Right	<code>\dot{x}</code> , <code>\ddot{x}</code> — exit with Right
$\acute{x}$ $\grave{x}$ $\check{x}$ $\ddot{x}$ $\underset{\cdot}{x}$ <i>x accents</i>	<code>\acute</code> +Enter <code>x</code> Right ...	<code>\acute{x}</code> , <code>\b{o}</code> — under-accents <code>\b</code> / <code>\c</code> / <code>\d</code> work; each accent's arg is an open block (Right to exit)
arrows over	<code>\overrightarrow</code> <code>\overleftarrow</code> <code>\overleftrightarrow</code> <code>\overarc</code> +Enter	<code>\overrightarrow{AB}</code>
$\overbrace{x+y}^n$ $\underbrace{x+y}_n$	<code>\overbrace</code> +Enter <code>x+y</code> Right <code>^n</code>	<code>\overbrace{x+y}^n</code>
$\cancel{x}$	<code>\cancel</code> +Enter <code>x</code> Right	<code>\cancel{x}</code>
$\boxed{x=1}$	<code>\boxed</code> +Enter <code>x=1</code>	<code>\boxed{x=1}</code>
$\underset{a}{b}$ $\overset{a}{b}$	<code>\underset</code> +Enter <code>a</code> Tab <code>b</code>	two-arg: <code>{under}{base}</code> , <code>{over}{base}</code>

`\widehat` works but serializes `\hat` ; `\bar` serializes `\overline` .

Fonts, text, spacing

Want	Type	Get
$\mathbb{R}$	<code>\R</code> +Enter, <code>reals</code> , or unicode $\mathbb{R}$	<code>\mathbb{R}</code> — also <code>\Z</code> <code>\N</code> <code>\Q</code> <code>\C</code> <code>\P</code> <code>\H</code> , words <code>naturals</code> <code>integers</code> <code>rationals</code> <code>complexes</code> <code>quaternions</code> , unicode <code>\Z</code> <code>\N</code> <code>\Q</code> <code>\C</code>
$\mathbb{F}_p$	—	<code>\mathbb{F}</code> +Enter drops its wrapper and unicode $\mathbb{F}$ is dropped entirely — <code>\mathbb{F}</code> is unwritable (see missing-features)
$\mathcal{L}$ $\mathcal{g}$ $\mathcal{L}$	<code>\mathcal</code> +Enter <code>L</code> Right etc.	<code>\mathcal{L}</code> , <code>\mathfrak{g}</code> , <code>\mathscr{L}</code>
$\mathbf{x}$ $\mathbf{x}$ $\mathbf{x}$ $\mathbf{x}$ $\mathbf{x}$	<code>\mathbf</code> +Enter <code>x</code> etc.	kept — <code>\bf</code> +Enter also works
custom upright operator	<code>\mathop</code> +Enter <code>tr</code> Right <code>(A)</code>	<code>\mathop{tr}\left(A\right)</code> — the workaround for dropped <code>\operatorname</code>
text in math	<code>\text</code> +Enter <code>if x>0</code> Right	<code>\text{if x>0}</code> — spaces allowed; Right exits
bold/italic text	<code>\textbf</code> +Enter / <code>\textit</code> +Enter	<code>\textbf{note}</code> , <code>\textit{note}</code>
spacing	<code>\quad</code> <code>\qquad</code> <code>\,</code> , <code>\;</code> <code>\!</code> <code>\</code>	<code>\quad</code> <code>\qquad</code> <code>\,</code> , <code>\;</code> <code>\!</code> all work now (<code>\!</code> = negative thin space)
measured spacing	<code>\hspace</code> +Enter <code>1em</code> Right / <code>\kern</code> +Enter <code>5pt</code> Right	<code>\hspace{1em}</code> , <code>\kern{5pt}</code>
invisible height	<code>\phantom</code> +Enter <code>x</code> Right	<code></code>

Want	Type	Get
dots	<code>\dots \cdots \vdots \ddots</code> <code>\ldots \hdots \dotsb \dotsi</code>	all serialize

`\mathbb{F}` freeform, `\operatorname`, `\textcolor` are still dead — see MISSING-INPUT-FEATURES.md. Text-style words `textnormal` `textrm` `textsf` `texttt` `textbf` `textit` `textmd` `textsc` `textsl` `textup` `emph` `italic` `italics` `bold` `strong` `tt` `sf` `uppercase` `lowercase` are registered.

Matrices, piecewise, environments

Two ways in, both fine. The **name shortcuts** (`\pmatrix`, `\cases`, `\aligned`, `\gathered`, `\matrix`, `\bmatrix`, `\Bmatrix`, `\vmatrix`, `\Vmatrix`, `\smallmatrix`, `\displaylines` +Enter) insert a blank grid with the caret in the **first cell** — prefer these. Canonical aliases resolve to the same grids: `\align \split \eqnarray \eqalign` → `aligned`, `\gather \multline \flalign` → `gathered`, `\equation` → single-cell `gathered`. Or type `\begin{env}` **literally** — a blank grid is inserted (2 cols × 3 rows) with the caret in **row 2, col 1** — press **Up** to reach row 1.

The spec-arg environments carry their argument inline: `\array{cc}`, `\subarray{c}`, `\tabular{ll}`, `\alignat{2}`, `\alignedat{n}` — or the bare `\array` +Enter, which opens a pending `{ }` input where `}`, Enter, or Tab applies the spec. A typed `\begin{array}{cc}` routes through the same input.

Want	Type	Get
2×2 parens matrix	<code>\pmatrix</code> +Enter a Tab b Enter c Tab d	<code>\begin{pmatrix}a&b\\c&d\end{pmatrix}</code> (after filling)
same, literal env	<code>\begin{pmatrix}</code> +Enter Up a Tab b ...	works — caret starts row 2, Up reaches row 1
det matrix	det <code>\vmatrix</code> +Enter ...	<code>\det\begin{vmatrix}...</code> — <code>\vmatrix</code> = det bars, <code>\Vmatrix</code> = double bars, <code>\bmatrix</code> = <code>[]</code> , <code>\Bmatrix</code> = <code>{ }</code> , <code>\matrix</code> = bare
piecewise $f(x) = \begin{cases} x^2 & x > 0 \\ 0 & x < 0 \end{cases}$	<code>f(x)= \cases</code> +Enter <code>x^2</code> Right Tab <code>x>0</code> Enter <code>0</code> Tab <code>x<0</code>	<code>\begin{cases}x^{x>0}\0&x<0\end{cases}</code> — col 2 is the condition; Right exits the sup block <i>then</i> Tab moves to it. <code>\begin{cases}</code> +Enter also works
$\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}$	<code>\smallmatrix</code> +Enter	works — <code>\begin{smallmatrix}</code> +Enter too
$\begin{array}{cc} a & b \\ c & d \end{array}$	<code>\array{cc}</code> +Enter	works — the <code>{cc}</code> spec applies; <code>\begin{array}{cc}</code> typed literally lands in the same arg input
<code>\begin{subarray}{c}</code>	<code>\subarray{c}</code> +Enter	works — <code>{c}</code> spec applies
aligned work	<code>\begin{aligned}</code> +Enter x Tab <code>=1</code> Enter ...	<code>\begin{aligned}x&=1\\...</code> — col1 = left side, col2 = <code>=rhs</code> . Enter adds a row keeping the column — arrow Left back to col 1 to start each row. <code>&</code> types literal <code>\&</code> , never a column break
gathered lines	<code>\gathered</code> +Enter x=1 Enter y=2	<code>\begin{gathered}x=1\\y=2\end{gathered}</code> — <code>\gather</code> <code>\multline</code> <code>\flalign</code> shortcuts and <code>\begin{gather}</code> <code>\begin{equation}</code> <code>\begin{multline}</code> <code>\begin{flalign}</code> all resolve the same
more aligned aliases	<code>\aligned</code> +Enter; <code>\align</code> <code>\split</code> <code>\eqnarray</code> <code>\eqalign</code>	resolve to <code>\begin{aligned}</code> ; <code>\alignat{2}</code> / <code>\alignedat{2}</code> keep their <code>{n}</code> column spec

- **Tab/Right move to the next cell**; arrows move cell-to-cell; **Enter inserts a row below** keeping the column; **Shift+Space adds a column**. Typing `&` inside a cell inserts literal `\&`.
- **The only keyboard exit is Ctrl+End** (jump to field end, landing at baseline after the env) — Right at the last cell stays inside, Down on the last row stays inside.

- Typing `\pmatrix{a&b\\c&d}` inline mangles — the `&/brace` chars are not cell syntax while typing.

Multi-line work

- **Enter at top level** wraps the cell in `\displaylines{...}` and adds a row: `x=1` Enter `y=2` → `\displaylines{x=1\\ y=2}`. More Enters → more rows (`a` Enter `b` Enter `c` → `\displaylines{a\\ b\\ c}`).
- Enter inside an incomplete construct (e.g. mid- `\frac`) snaps the caret out to the atom first, then breaks the line — the incomplete atom is left as-is.
- `\displaylines` +Enter as a command also inserts the block.
- For real aligned/gathered structure use `\begin{aligned}` / `\begin{gathered}` (above) — `\displaylines` rows are simpler but have no alignment columns.
- **Shift+Enter** → **new cell** (verified: field count grows, new cell gets focus). Backspace/Delete on a blank cell deletes the cell.
- Navigation: `Home` / `End` are block-local; `Ctrl+Home` / `Ctrl+End` go to field start/end. Up/Down at a cell's vertical edge hop to the neighboring cell (`move-out`).

Special characters and unicode

[illegible]

Worked examples from the theses

All verified end-to-end in the app. Right presses are the block exits that matter.

Expression	Keystrokes	Result
$f: S^1 \rightarrow R^3$ (Buller)	f:S^1 Right \to +Enter R^3	f:S^{1}\to R^{3}
$H: Y \times [0, 1] \rightarrow Y$	H:Y\times +Enter [0,1]\to +Enter Y	H:Y\times\left[0,1\right]\to Y

Expression	Keystrokes	Result
$L = K_1 \cup K_2 \cup \dots \cup K_n \subset S^3$ (Buller)	L=K_1 Right \cup +Enter K_2 Right \cup +Enter \cdots +Enter \cup +Enter K_n Right \subset +Enter S^3	$L=K_{\{1\}}\cup K_{\{2\}}\cup\cdots\cup K_{\{n\}}\subset S^{\{3\}}$
$G(T_{p,q}) = \langle x, y \mid x^p = y^q \rangle$ (Salkinder)	G(T_p,q Right)=\langle +Enter x,y\mid +Enter x^p Right =y^q Ctrl+End	$G\left(T_{\{p,q\}}\right)=\left\langle x,y\mid x^{\{p\}}=y^{\{q\}}\right\rangle$ — do not type \rangle (see rule 3)
$D_{2n} = \langle s, t \mid s^2 = t^n = 1, sts^{-1} = t^{-1} \rangle$ (Salkinder)	D_2n Right =\langle +Enter s,t\mid +Enter s^2=t^n=1,sts^{-1}=t^{-1} Right Ctrl+End	$D_{\{2n\}}=\left\langle s,t\mid s^{\{2\}}=t^{\{n\}}=1,sts^{\{-1\}}=t^{\{-1\}}\right\rangle$
$\frac{Z}{p} * \frac{Z}{q}$	\Z +Enter /p Right \ast +Enter \Z +Enter /q	$\frac{\mathbb{Z}}{p}\ast\frac{\mathbb{Z}}{q}$ — for inline Z/p use \slash
$g_4(K) = \min\{g(S) \mid S \subset B^4\}$ (Buller)	g_4 Right (K)=\min\{g(S)\mid +Enter S\subset +Enter B^4 Right }	$g_{\{4\}}\left(K\right)=\min\left\{g\left(S\right)\mid S\subset B^{\{4\}}\right\}$
$t^{-1}V(L+) - tV(L-) - (t^{1/2} - t^{-1/2})V(L_0) = 0$ (Buller, Alexander poly.)	t^{-1} Right V(L+) - tV(L-) - (t^{1/2} Right Right -t^{-1/2} Right Right)V(L_0 Right)=0	$t^{\{-1\}}V\left(L+\right)-tV\left(L-\right)-\left(t^{\{\frac{1}{2}\}}-t^{\{-\frac{1}{2}\}}\right)V\left(L_{\{0\}}\right)=0$
$x' = F(x, \mu)$ (Fogelson)	x'=F(x,\mu)	$x'=F\left(x,\mu\right)$
$\mathrm{tr}(DF(x, \mu))$ (Fogelson)	\mathop +Enter tr Right (DF(x,\mu))	$\mathop{\mathrm{tr}}\left(DF\left(x,\mu\right)\right)$ — upright tr
$\lim_{t \rightarrow 0} x(t) = x$ (Fogelson)	\lim _t->0 Right x(t)=x	$\lim_{t\rightarrow 0}x\left(t\right)=x$
$\Re \lambda_i(DF)$ (Fogelson, eigenvalues)	\Re +Enter \lambda +Enter _i Right (DF)	$\Re\lambda_{\{i\}}\left(DF\right)$
$M_t := f^{-1}((-\infty, t])$ (Fogelson, sublevel set)	M_t Right :=f^{-1} Right ((- \infty +Enter ,t])	$M_{\{t\}}:=f^{\{-1\}}\left(\left(-\infty,t\right]\right)$ — := literal works, or \assign +Enter
$D^k \times D^{n-k}$ (Salkinder, handles)	D^k Right \times +Enter D^{n-k}	$D^{\{k\}}\times D^{\{n-k\}}$
$\tilde{H}_i(S^n \setminus A)$ (Salkinder, homology)	\tilde +Enter H_i Right (S^n Right \setminus +Enter A)	$\tilde{H}_{\{i\}}\left(S^{\{n\}}\setminus A\right)$
$\rho : G \rightarrow \mathrm{Aut}(V)$ (Salkinder)	\rho +Enter :G\to +Enter \mathop +Enter Aut Right (V)	$\rho:G\rightarrow\mathop{\mathrm{Aut}}\left(V\right)$ — upright Aut via \mathop
$w(L) = \sum_{c \in C} \mathit{sgn}(c)$ (Buller, writhe)	w(L)=\sum +Enter c\in +Enter C Right Right sgn(c)	$w\left(L\right)=\sum_{c\in C}\mathit{sgn}\left(c\right)$ — typing crossings verbatim traps on sin ; use \text{crossings}
$x + y \equiv 2z \pmod{3}$ (Buller)	x+y\equiv +Enter 2z \pmod +Enter 3	$x+y\equiv 2z\pmod{3}$ — \pmod / \bmod / \mod all work